

Python Chapter 3

Notes

Lists in Python :

Lists are used to store multiple items in a single variable.

Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.

The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

List Items

List items are ordered, changeable, and allow duplicate values.

List items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

If you add new items to a list, the new items will be placed at the end of the list.

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates

Since lists are indexed, lists can have items with the same value:

Accessing Values in Lists

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];  
list2 = [1, 2, 3, 4, 5, 6, 7 ];  
print "list1[0]: ", list1[0]  
print "list2[1:5]: ", list2[1:5]
```

Updating Lists

You can update single or multiple elements of lists by giving the slice on the left-hand side of the assignment operator, and you can add to elements in a list with the append() method. For example –

```
list = ['physics', 'chemistry', 1997, 2000];  
print "Value available at index 2 : "  
print list[2]  
list[2] = 2001;  
print "New value available at index 2 : "
```

```
print list[2]
```

Delete List Elements

To remove a list element, you can use either the `del` statement if you know exactly which element(s) you are deleting or the `remove()` method if you do not know. For example –

```
list1 = ['physics', 'chemistry', 1997, 2000];  
print list1  
del list1[2];  
print "After deleting value at index 2 : "  
print list1
```

Basic List Operations

Lists respond to the `+` and `*` operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

In fact, lists respond to all of the general sequence operations we used on strings in the prior chapter.

Expression	Results	Descriptions
<code>Len([1,2,3])</code>	3	Length
<code>[1,2,3] + [4,5,6]</code>	<code>[1,2,3,4,5,6]</code>	Concat
<code>['Hi!'] * 4</code>	<code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code>	Repeat
<code>3 in [1,2,3]</code>	True	Membership

for x in [1, 2, 3]: print x,	1 2 3	Iteration
------------------------------	-------	-----------

Built-in Functions

1. Compare

Description

Python list method **cmp()** compares elements of two lists.

Syntax

Following is the syntax for **cmp()** method –

```
cmp(list1, list2)
```

Parameters

- **list1** – This is the first list to be compared.
- **list2** – This is the second list to be compared.

Return Value

If elements are of the same type, perform the compare and return the result. If elements are different types, check to see if they are numbers.

- If numbers, perform numeric coercion if necessary and compare.
- If either element is a number, then the other element is "larger" (numbers are "smallest").
- Otherwise, types are sorted alphabetically by name.

If we reached the end of one of the lists, the longer list is "larger." If we exhaust both lists and share the same data, the result is a tie, meaning that 0 is returned.

Example

The following example shows the usage of `cmp()` method.

```
list1, list2 = [123, 'xyz'], [456, 'abc']  
print cmp(list1, list2)  
print cmp(list2, list1)  
list3 = list2 + [786];  
print cmp(list2, list3)
```

2. Length

Description

Python list method **`len()`** returns the number of elements in the *list*.

Syntax

Following is the syntax for **`len()`** method –

```
len(list)
```

Parameters

- **list** – This is a list for which number of elements to be counted.

Return Value

This method returns the number of elements in the list.

Example

The following example shows the usage of `len()` method.

```
list1, list2 = [123, 'xyz', 'zara'], [456, 'abc']  
print "First list length : ", len(list1)  
print "Second list length : ", len(list2)
```

When we run above program, it produces following result –

```
First list length : 3  
Second list length : 2
```

3. Maximum

Description

Python list method **max** returns the elements from the *list* with maximum value.

Syntax

Following is the syntax for **max()** method –

```
max(list)
```

Parameters

- **list** – This is a list from which max valued element to be returned.

Return Value

This method returns the elements from the list with maximum value.

Example

The following example shows the usage of max() method.

```
list1, list2 = [123, 'xyz', 'zara', 'abc'], [456, 700, 200]
```

```
print "Max value element : ", max(list1)
print "Max value element : ", max(list2)
```

When we run above program, it produces following result –

Max value element : zara

Max value element : 700

4. Minimum

Description

Python list method **min()** returns the elements from the *list* with minimum value.

Syntax

Following is the syntax for **min()** method –

```
min(list)
```

Parameters

- **list** – This is a list from which min valued element to be returned.

Return Value

This method returns the elements from the list with minimum value.

Example

The following example shows the usage of min() method.

```
list1, list2 = [123, 'xyz', 'zara', 'abc'], [456, 700, 200]
```

```
print "min value element : ", min(list1)
print "min value element : ", min(list2)
```

When we run above program, it produces following result –

```
min value element : 123
min value element : 200
```

5. List(Sequence)

Description

Python list method **list()** takes sequence types and converts them to lists. This is used to convert a given tuple into list.

Note – Tuple are very similar to lists with only difference that element values of a tuple can not be changed and tuple elements are put between parentheses instead of square bracket.

Syntax

Following is the syntax for **list()** method –

```
list( seq )
```

Parameters

- **seq** – This is a tuple to be converted into list.

Return Value

This method returns the list.

Example

The following example shows the usage of list() method.

```
aTuple = (123, 'xyz', 'zara', 'abc');  
aList = list(aTuple)  
print "List elements : ", aList
```

When we run above program, it produces following result –

List elements : [123, 'xyz', 'zara', 'abc']

Tuples in Python :

A tuple is a collection of objects which ordered and immutable. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5 );  
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
tup2 = (1, 2, 3, 4, 5, 6, 7 );  
print "tup1[0]: ", tup1[0];  
print "tup2[1:5]: ", tup2[1:5];
```

When the above code is executed, it produces the following result –

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```

Updating Tuples

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);  
tup2 = ('abc', 'xyz');
```

```
# Following action is not valid for tuples
```

```
# tup1[0] = 100;
```

So let's create a new tuple as follows

```
tup3 = tup1 + tup2;  
print tup3;
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example –

```
tup = ('physics', 'chemistry', 1997, 2000);  
print tup;  
del tup;  
print "After deleting tup : ";  
print tup;
```

This produces the following result. Note an exception raised, this is because after **del tup** tuple does not exist any more –

```
('physics', 'chemistry', 1997, 2000)
```

After deleting tup :

Traceback (most recent call last):

File "test.py", line 9, in <module>

```
print tup;
```

NameError: name 'tup' is not defined

Basic Tuples Operations

Tuples respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new tuple, not a string..

Expression	Results	Descriptions
Len((1,2,3))	3	Length
(1,2,3) + (4,5,6)	(1,2,3,4,5,6)	Concat
('Hi!') * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repeat
3 in (1,2,3)	True	Membership
for x in (1, 2, 3): print x,	1 2 3	Iteration

Built-in Tuple Functions

1. Compare

Description

Python tuple method **cmp()** compares elements of two tuples.

Syntax

Following is the syntax for **cmp()** method –

```
cmp(tuple1, tuple2)
```

Parameters

- **tuple1** – This is the first tuple to be compared
- **tuple2** – This is the second tuple to be compared

Return Value

If elements are of the same type, perform the compare and return the result. If elements are different types, check to see if they are numbers.

- If numbers, perform numeric coercion if necessary and compare.
- If either element is a number, then the other element is "larger" (numbers are "smallest").
- Otherwise, types are sorted alphabetically by name.

If we reached the end of one of the tuples, the longer tuple is "larger." If we exhaust both tuples and share the same data, the result is a tie, meaning that 0 is returned.

Example

The following example shows the usage of cmp() method.

```
tuple1, tuple2 = (123, 'xyz'), (456, 'abc')
print cmp(tuple1, tuple2)
print cmp(tuple2, tuple1)
tuple3 = tuple2 + (786,);
print cmp(tuple2, tuple3)
```

When we run above program, it produces following result –

```
-1
1
-1
```

2. Length

Description

Python tuple method **len()** returns the number of elements in the tuple.

Syntax

Following is the syntax for **len()** method –

```
len(tuple)
```

Parameters

- **tuple** – This is a tuple for which number of elements to be counted.

Return Value

This method returns the number of elements in the tuple.

Example

The following example shows the usage of len() method.

```
tuple1, tuple2 = (123, 'xyz', 'zara'), (456, 'abc')  
print "First tuple length : ", len(tuple1)  
print "Second tuple length : ", len(tuple2)
```

When we run above program, it produces following result –

First tuple length : 3

Second tuple length : 2

3. Maximum

Description

Python tuple method **max** returns the elements from the *tuple* with maximum value.

Syntax

Following is the syntax for **max()** method –

```
max(tuple)
```

Parameters

- **tuple**– This is a tuple from which max valued element to be returned.

Return Value

This method returns the elements from the tuple with maximum value.

Example

The following example shows the usage of max() method.

```
Tuple1, tuple2 = (123, 'xyz', 'zara', 'abc'), (456, 700, 200)
print "Max value element : ", max(tuple1)
print "Max value element : ", max(tuple2)
```

When we run above program, it produces following result –

Max value element : zara

Max value element : 700

4. Minimum

Description

Python tuple method **min()** returns the elements from the *tuple* with minimum value.

Syntax

Following is the syntax for **min()** method –

```
min(tuple)
```

Parameters

- **tuple**– This is a list from which min valued element to be returned.

Return Value

This method returns the elements from the tuple with minimum value.

Example

The following example shows the usage of min() method.

```
Tuple1, tuple2 = (123, 'xyz', 'zara', 'abc'), (456, 700, 200)
print "min value element : ", min(tuple1)
print "min value element : ", min(tuple2)
```

When we run above program, it produces following result –

```
min value element : 123
```

```
min value element : 200
```


Sets:

Mathematically a set is a collection of items not in any particular order. A Python set is similar to this mathematical definition with below additional conditions.

- The elements in the set cannot be duplicates.
- The elements in the set are immutable(cannot be modified) but the set as a whole is mutable.
- There is no index attached to any element in a python set. So they do not support any indexing or slicing operation.

Set Operations

The sets in python are typically used for mathematical operations like union, intersection, difference and complement etc. We can create a set, access it's elements and carry out these mathematical operations as shown below.

Creating a set

A set is created by using the set() function or placing all the elements within a pair of curly braces.

```
Days=set(["Mon","Tue","Wed","Thu","Fri","Sat","Sun"])
```

```
Months={"Jan","Feb","Mar"}
```

```
Dates={21,22,17}
```

```
print(Days)
```

```
print(Months)
```

```
print(Dates)
```

When the above code is executed, it produces the following result. Please note how the order of the elements has changed in the result.

```
set(['Wed', 'Sun', 'Fri', 'Tue', 'Mon', 'Thu', 'Sat'])  
set(['Jan', 'Mar', 'Feb'])  
set([17, 21, 22])
```

Accessing Values in a Set

We cannot access individual values in a set. We can only access all the elements together as shown above. But we can also get a list of individual elements by looping through the set.

```
Days=set(["Mon","Tue","Wed","Thu","Fri","Sat","Sun"])
```

```
for d in Days:  
    print(d)
```

When the above code is executed, it produces the following result.

```
Wed  
Sun  
Fri  
Tue  
Mon  
Thu  
Sat
```

Adding Items to a Set

We can add elements to a set by using add() method. Again as discussed there is no specific index attached to the newly added element.

```
Days=set(["Mon","Tue","Wed","Thu","Fri","Sat"])
```

```
Days.add("Sun")
```

```
print(Days)
```

When the above code is executed, it produces the following result.

```
set(['Wed', 'Sun', 'Fri', 'Tue', 'Mon', 'Thu', 'Sat'])
```

Removing Item from a Set

We can remove elements from a set by using discard() method. Again as discussed there is no specific index attached to the newly added element.

```
Days=set(["Mon","Tue","Wed","Thu","Fri","Sat"])
```

```
Days.discard("Sun")
```

```
print(Days)
```

When the above code is executed, it produces the following result.

```
set(['Wed', 'Fri', 'Tue', 'Mon', 'Thu', 'Sat'])
```

Union of Sets

The union operation on two sets produces a new set containing all the distinct elements from both the sets. In the below example the element “Wed” is present in both the sets.

```
DaysA = set(["Mon","Tue","Wed"])
DaysB = set(["Wed","Thu","Fri","Sat","Sun"])
AllDays = DaysA|DaysB
print(AllDays)
```

When the above code is executed, it produces the following result. Please note the result has only one “wed”.

```
set(['Wed', 'Fri', 'Tue', 'Mon', 'Thu', 'Sat'])
```

Intersection of Sets

The intersection operation on two sets produces a new set containing only the common elements from both the sets. In the below example the element “Wed” is present in both the sets.

```
DaysA = set(["Mon","Tue","Wed"])
DaysB = set(["Wed","Thu","Fri","Sat","Sun"])
AllDays = DaysA & DaysB
print(AllDays)
```

When the above code is executed, it produces the following result. Please note the result has only one “wed”.

```
set(['Wed'])
```

Difference of Sets

The difference operation on two sets produces a new set containing only the elements from the first set and none from the second set. In the below example

the element “Wed” is present in both the sets so it will not be found in the result set.

```
DaysA = set(["Mon","Tue","Wed"])
DaysB = set(["Wed","Thu","Fri","Sat","Sun"])
AllDays = DaysA - DaysB
print(AllDays)
```

When the above code is executed, it produces the following result. Please note the result has only one “wed”.

```
set(['Mon', 'Tue'])
```

Compare Sets

We can check if a given set is a subset or superset of another set. The result is True or False depending on the elements present in the sets.

```
DaysA = set(["Mon","Tue","Wed"])
DaysB = set(["Mon","Tue","Wed","Thu","Fri","Sat","Sun"])
SubsetRes = DaysA <= DaysB
SupersetRes = DaysB >= DaysA
print(SubsetRes)
print(SupersetRes)
```

When the above code is executed, it produces the following result.

True

True

Dictionary:

Python Dictionary is used to store the data in a key-value pair format. The dictionary is the data type in Python, which can simulate the real-life data arrangement where some specific value exists for some particular key. It is the mutable data-structure. The dictionary is defined into element Keys and values.

- Keys must be a single element
- Value can be any type such as list, tuple, integer, etc.

In other words, we can say that a dictionary is the collection of key-value pairs where the value can be any Python object. In contrast, the keys are the immutable Python object, i.e., Numbers, string, or tuple.

Creating the dictionary

The dictionary can be created by using multiple key-value pairs enclosed with the curly brackets {}, and each key is separated from its value by the colon (:). The syntax to define the dictionary is given below.

Syntax:

1. Dict = {"Name": "Tom", "Age": 22}

In the above dictionary **Dict**, The keys **Name** and **Age** are the string that is an immutable object.

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

Example

Get the value of the "model" key:

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
}  
x = thisdict["model"]
```

There is also a method called `get()` that will give you the same result:

Example

Get the value of the "model" key:

```
x = thisdict.get("model")
```

Get Keys

The `keys()` method will return a list of all the keys in the dictionary.

Example

Get a list of the keys:

```
x = thisdict.keys()
```

The list of the keys is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the keys list.

Example

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}  
  
x = car.keys()  
  
print(x) #before the change  
  
car["color"] = "white"  
  
print(x) #after the change
```